Comprehensive Guide to `constexpr` in C++

Detailed Notes

Contents

1 Introduction to `constexpr`

The `constexpr` specifier, introduced in C++11, declares that it is possible to evaluate the value of the function or variable at compile time. It serves as a powerful tool for template metaprogramming, performance optimization, and shifting computation from run-time to compile-time.

While `const` specifies that an object's value cannot be modified after initialization, `constexpr` goes a step further: it requires the object to be initialized with a value known during compilation.

2 `constexpr` Variables

When applied to a variable, `constexpr` implies `const` but enforces compile-time initialization.

2.1 Requirements for `constexpr` Variables

- Its type must be a **LiteralType** (e.g., scalar types, references, arrays of literal types, or classes with a trivial destructor and at least one `constexpr` constructor).
- It must be immediately initialized.
- The initialization expression must be a **constant expression**.

```
const int runtime_val = get_value(); // Valid: const but initialized at
    runtime
constexpr int c1 = 42;               // Valid: 42 is a constant
    expression
constexpr int c2 = c1 + 10;          // Valid: c1 is known at compile-
    time

// constexpr int c3 = get_value();   // Error: get_value() is not
    constexpr
```

3 `constexpr` Functions

A `constexpr` function is a function that *can* be evaluated at compile-time if all its arguments are constant expressions. If called with non-constant arguments, it executes at run-time like a normal function (unless evaluated in a context that requires a constant expression).

3.1 Evolution of `constexpr` Functions

3.1.1 C++11: The Strict Era

In C++11, `constexpr` functions were heavily restricted:

- The function body could basically only contain a single `return` statement.
- No local variables, loops, or `if` statements were allowed (recursion and ternary operators were used instead).

```
// C++11 style constexpr factorial
constexpr int factorial(int n) {
    return n <= 1 ? 1 : (n * factorial(n - 1));
}
```

3.1.2 C++14: The Relaxation

C++14 drastically relaxed the rules. `constexpr` functions could now contain:

- Local variable declarations (must be initialized).
- Conditional statements (`if`, `switch`).
- Loops (`for`, `while`, `do-while`).
- Mutation of objects whose lifetime began within the constant expression evaluation.

```
// C++14 style constexpr factorial
constexpr int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; ++i) {
        result *= i;
    }
    return result;
}
```

3.1.3 C++17: Lambdas and `if constexpr`

- Lambdas became implicitly `constexpr` if their body satisfies the requirements.
- Introduction of `if constexpr` (detailed in Section 5).

3.1.4 C++20: Virtual Functions, Allocations, and Try/Catch

C++20 massively expanded what is possible:

- Virtual functions can now be `constexpr`.
- Dynamic memory allocation (`new`, `delete`) is allowed, provided the memory is deallocated before the compile-time evaluation finishes.
- `std::string` and `std::vector` can be used in `constexpr` contexts (transient allocations).
- `try`/`catch` blocks are allowed (though throwing an exception terminates compile-time evaluation).

4 `constexpr` Constructors and Classes

You can create `constexpr` objects of custom class types if they have a `constexpr` constructor.

```
struct Point {
    double x, y;

    // constexpr constructor
    constexpr Point(double x, double y) : x(x), y(y) {}

    // constexpr member function
    constexpr double getX() const { return x; }
};

constexpr Point p(10.5, 20.0);
constexpr double px = p.getX(); // Evaluated at compile-time
```

5 `if constexpr` (C++17)

`if constexpr` is a compile-time conditional statement. The compiler completely discards the branch that evaluates to false. This is extremely useful for template metaprogramming, replacing SFINAE (Substitution Failure Is Not An Error) and tag dispatching.

```
template <typename T>
auto get_value(T t) {
    if constexpr (std::is_pointer_v<T>) {
        return *t; // Only compiled if T is a pointer
    } else {
        return t;  // Only compiled if T is not a pointer
    }
}
```

6 C++20 Additions: `consteval` and `constinit`

6.1 `consteval` (Immediate Functions)

While `constexpr` functions *can* be evaluated at compile-time, they can also fall back to run-time. If you want to **guarantee** that a function is evaluated at compile-time, use `consteval` (C++20).

```
consteval int sqr(int n) {
    return n * n;
}

constexpr int r = sqr(10); // OK
int x = 10;
// int r2 = sqr(x);        // ERROR: x is not a constant expression
```

6.2 `constinit`

`constinit` ensures that a variable has static initialization (initialized at compile-time), thereby solving the **Static Initialization Order Fiasco**. Unlike `constexpr`, `constinit` does not imply `const`, so the variable can be mutated later at run-time.

```
constexpr int get_init() { return 42; }

constinit int global_counter = get_init(); // Guaranteed compile-time
    init

int main() {
    global_counter++; // Valid! It is not const.
}
```

7 Summary of Limitations (What you CANNOT do)

Even in modern C++, `constexpr` evaluation has boundaries:

- **No side effects:** You cannot do I/O (e.g., `std::cout`) during compile-time.
- **No `reinterpret_cast`:** Type punning via `reinterpret_cast` is strictly forbidden.
- **No uninitialized variables:** Every local variable must be initialized.

- **No `goto` or labels:** (Though loops are fine).
- **Pointers to arbitrary memory:** You cannot cast an integer to a pointer and dereference it.